

Project Email template maintenance

Q:

Ik wil een uitbreiding maken van ons adminj 7 applicatie en een tabel waar medere email layouts instaan in html layout en in text format in en 3 talen en, nl, en fr. Wat ik nodig heb is een een extractie routine via dropdown box om een taal en een email nr te kiezen en ook de kiezen of ik de html versie of de textversie wil bewerken. Dit moet adminjs conform via een handelbaar procedure. De email moet je kunnen updaten en terug in de tabel zetten.

A:

Dit is een uitdagende maar zeer relevante functionaliteit voor een administratief systeem (AdminJS). Omdat AdminJS sterk gebaseerd is op React en werkt met resources die direct gekoppeld zijn aan je database-modellen, moeten we dit niet benaderen als een simpele "tabel", maar als een **Custom Component** of een **Action** binnen de AdminJS architectuur.

Hier is het stappenplan om dit technisch te implementeren:

1. De Datastructuur (Database Model)

Om dit goed te beheren, heb je een tabel nodig die niet alleen de tekst bevat, maar ook de metadata over welke taal en welk type layout het is.

Aanbevolen model (EmailTemplate):

- `id`: UUID / Primary Key
- `email_slug`: Unieke naam (bijv: `order_confirmation`)
- `language`: Enum (`nl`, `fr`, `en`)
- `type`: Enum (`html`, `text`)
- `content`: Text/LongText field waarin de HTML-code of de platte tekst staat.

Tip: Je kunt ook één rij per template maken met kolommen voor `NL_HTML`, `NL_TEXT`, `FR_HTML`, etc., maar het bovenstaande "row-per-variant" model is veel schaalbaarder.

2. De Logica in AdminJS (De 'Handelbare Procedure')

Je wilt geen standaard CRUD (Create, Read, Update, Delete) waarbij je telkens een nieuwe rij aanmaakt met alle talen tegelijk. Je wilt een **interactieve interface**. Dit lossen we op via een **Custom Page** of een **Custom Action**.

Stap A: De Interface (Frontend - React)

Je moet een custom React component maken die binnen de AdminJS UI wordt geladen. Deze component bevat:

1. **Dropdown 1 (Email Selectie)**: Haalt alle unieke `email_slugs` op uit je database.
2. **Dropdown 2 (Taal Selectie)**: Hardcoded opties: NL, FR, EN.
3. **Toggle/Radio Buttons**: Keuze tussen "HTML Editor" of "Text Editor".

Stap B: De Editor

- Voor **Text**: Een eenvoudige `<textarea>`.
- Voor **HTML**: Gebruik een library zoals `react-simple-code-editor` of (indien gewenst) een WYSIWYG editor zoals `CKEditor` of `Quill`, maar voor email is een code-editor vaak veiliger om CSS-fouten te voorkomen.

3. De Workflow van de "Extractie & Update" routine

Zo ziet het proces er technisch uit:

1. **Selectie**: Gebruiker kiest `[Welcome Email] + [FR] + [HTML]`.
2. **Fetch (De Extractie)**: Bij de selectie triggert een `useEffect` hook in je React component een API-call naar je backend met parameters: `{ slug: 'welcome', lang: 'fr', type: 'html' }`.
3. **Display**: De data wordt opgehaald en de editor vult zich met de bestaande HTML code.
4. **Bewerken**: De gebruiker past de tekst aan in de editor.
5. **Opslaan (De Update)**: Bij het klikken op "Opslaan" stuurt de component een POST of PUT request naar je backend/controller met de nieuwe content.

4. Implementatie Strategie voor AdminJS

Omdat je spreekt over een "handelbare procedure conform AdminJS", raad ik de volgende implementatieroute aan:

1. **Maak een Custom Resource**: In plaats van de standaard `EmailTemplate` resource te gebruiken, maak je een `EmailManager` resource aan die niet direct naar een tabel linkt, maar fungeert als een "container" voor deze custom pagina.
2. **Gebruik AdminJS.Base.Component**:
 - Registreer een nieuwe component: `adminjs-components/EmailEditor`.
 - Voeg een `action` toe aan je resource met het type `page`.
3. **Backend API Route**: Je moet in je backend (bijv. NestJS, Express of LoopBack) een specifieke endpoint maken die de GET en PUT verzoeken van deze editor afhandelt. AdminJS's standaard CRUD is vaak te beperkt voor complexe logica zoals "zoek op slug én taal".

Samenvattend Architectuuroverzicht

Component	Technologie	Doel
Database	PostgreSQL/MySQL	Opslag van slug, lang, type en content.
Backend Controller	Node.js (Express)	De "Logic" die de specifieke variant zoekt en opslaat.
AdminJS Action	AdminJS API	Koppelt de knop in het menu aan je custom pagina.
Custom UI Component	React + CodeMirror	De dropdowns en de editor voor HTML/Text.

Wil je dat ik een specifiek code-voorbeeld uitwerk (bijvoorbeeld de SQL-structuur of het React-component voor de dropdowns)?

Q:

Wij blijven nu eerst een stappenplan maken om het later dan stap per stap om te zetten in code en integreren en testen. ik geef nu al te structuur en in code van de tabel EmailTemplates. De show mode van de adminjs geeft all een goed layout voor een email template in html en raw text format. In een eerste step is het misschien mogelijk die layout in een via de edit button te bereiken die nu onhandig voor edit is. Ik ga de 2 formaten toevoegen. DashboardEmail TemplatesEdit

=> Edit template geladen, show template geladen en json van de export meegegeven

A:

Dit is een uitstekende basis. De JSON laat zien dat de databasestructuur al perfect is voor het doel dat je voor ogen hebt: elke combinatie van `template (slug)`, `language`, en `eventId` vormt een unieke rij met zowel de HTML (`contentRich`) als de tekstversie (`contentPlain`).

Het probleem waar je nu tegenaan loopt, is de standaard AdminJS ervaring: om iets te wijzigen moet je naar de specifieke rij navigeren (wat onhandig is omdat er voor elke taal een aparte rij is) en dan pas in de edit-modus gaan.

Hieronder volgt het **Stappenplan** om dit op een professionele, schaalbare manier aan te pakken. We verdelen dit in vier fasen:

Fase 1: Analyse & Architectuur (Huidige fase)

Voordat we code schrijven, definiëren we de logische stappen van de nieuwe "Email Manager" interface.

- **Doel:** Een centrale plek creëren waar een admin via drie filters direct bij de juiste content komt.

- **De Interface Logica (User Flow):**

1. Gebruiker opent de "Email Template Manager".
 2. Stap 1: Kies een **Template Naam** (bijv: `registration`). De lijst wordt gefilterd op unieke `template` waarden uit de database.
 3. Stap 2: Kies een **Taal** (`nl`, `en`, `fr`).
 4. Stap 3: Kies het **Format** (HTML of Plain Text).
 5. *Resultaat:* De editor (Rich Text voor HTML, Plain Text voor tekst) wordt direct geladen met de relevante content uit die specifieke database-rij.
-

Fase 2: Backend Voorbereiding (API & Logica)

We moeten de backend zo inrichten dat de frontend makkelijk kan "zoeken" zonder alle data te downloaden.

1. **Nieuwe API Endpoints:** We hebben twee specifieke routes nodig voor de editor:
 - `GET /api/email-templates/fetch?template=registration&lang=nl&type=html`: Zoekt exact die ene rij op en stuurt alleen `subject` en `contentRich` terug.
 - `PUT /api/email-templates/update`: Ontvangt de nieuwe tekst en update de specifieke ID in de database.
 2. **Query Logica:** De backend moet zoeken op een combinatie van drie velden: `template`, `language` EN eventueel het `eventId`.
-

Fase 3: Frontend Implementatie (AdminJS Custom Component)

Dit is waar de "magie" gebeurt binnen de AdminJS UI. We bouwen een nieuwe pagina die niet simpelweg een tabel is, maar een **Tool**.

1. **De Selectors:** Bouw een React-component met drie `<select>` boxen of dropdowns (gebruik de AdminJS/MUI componenten voor een consistente look).
 2. **De Editor Integratie:**
 - Implementeer een `RichTextEditor` (zoals CKEditor of TinyMCE) die reageert op de "HTML" keuze.
 - Implementeer een standaard `<textarea>` voor de "Plain Text" keuze.
 3. **State Management:** Zorg dat zodra er een dropdown wordt veranderd, de `content` in het tekstveld direct wordt bijgewerkt (via de API call).
-

Fase 4: Integratie & Testing

Het samenbrengen van de onderdelen en het valideren van de data-integriteit.

1. **AdminJS Register:** De nieuwe custom pagina/action toevoegen aan je AdminJS options.
 2. **Validatie Test:**
 - Controleer of het wijzigen van HTML niet per ongeluk de Plain Text versie overschrijft (en vice versa).
 - Test of de subject ook correct wordt opgeslagen wanneer je een taal wisselt.
 3. **Edge Case Testing:** Wat gebeurt er als een vertaling voor een specifieke taal nog *niet* bestaat in de database? De tool moet dan automatisch een nieuwe rij kunnen aanmaken (optioneel).
-

Samenvattend Stappenplan voor uitvoering:

Stap	Actie	Resultaat
1	Backend Routes maken	Endpoints om specifiek te zoeken op template/taal.
2	Custom React Component bouwen	De dropdowns + de twee editors (HTML/Text).
3	AdminJS Koppeling	De knop "Email Manager" in het menu van AdminJS.
4	Testen & Refinement	Controleren of HTML-tags correct worden opgeslagen.

Zullen we beginnen met Stap 1: Het ontwerpen van de Backend API (de logica om die specifieke rij op te halen)?

Q:

Nog niet. Hier is nog de layout in show mode: Dashboard Email TemplatesShow Show

A:

Ik begrijp je punt volledig. Je wilt eigenlijk niet een compleet nieuwe interface bouwen, maar de **huidige "Edit" functionaliteit van AdminJS verbeteren** zodat deze direct bruikbaar is voor jouw doeleinden.

Het huidige probleem is dat de standaard edit pagina van AdminJS alle velden in één grote lijst laat zien (Subject, Content Rich, Content Plain, Language, Template). Dat is onhandig omdat je te veel ziet en moet zoeken tussen velden die je op dat moment niet wilt aanpassen.

Als we jouw voorstel volgen: **"De Edit mode verbeteren zodat deze direct de content in een editor-box geeft (zonder lijnen/beperkingen)"**, dan wordt het stappenplan als volgt aangepast:

De "Quick Win" Strategie (Verbetering van de bestaande Edit Mode)

In plaats van een nieuwe pagina, gaan we de `Edit` actie van AdminJS aanpassen met **Custom Components**. Hierdoor blijft de vertrouwde flow behouden, maar wordt de interface gericht op jouw behoefte.

1. Analyse van de huidige "unhandige" Edit Mode

Nu ziet de edit-mode er waarschijnlijk zo uit:

- Input veld voor Subject
- Textarea (simpel) voor Content Rich
- Textarea (simpel) voor Content Plain
- Dropdown voor Language
- Read-only/Dropdown voor Template

2. Het plan om dit "Handelbaar" te maken via de Edit Button:

Stap A: De UI 'Clean-up' in de Editor We gebruiken AdminJS components om de standaard tekstvelden te vervangen door specifieke editors, afhankelijk van wat de gebruiker nodig heeft.

- In plaats van twee grote tekstvakken onder elkaar, maken we een **"Toggle Switch"** bovenaan de Edit pagina: `[ ] Bewerk HTML | [X] Bewerk Text`.
- Als je op **HTML** klikt, verbergt AdminJS het Plain Text veld en toont hij een volledige Rich Text Editor (met de juiste formatting tools) die precies lijkt op wat je in "Show" ziet.
- Als je op **Text** klikt, verbergt hij de HTML editor en toont hij een clean tekstvak voor de `contentPlain`.

Stap B: De Interface Layout verbeteren (De "Show Mode Look") Je gaf aan dat de Show mode een goede layout heeft. In de Edit mode willen we diezelfde visuele rust, maar dan met actieve invoervelden.

- We halen de overbodige informatie (zoals `Created At` of `Id`) uit het edit-scherf zodat je alleen ziet wat relevant is: **Subject** en **Content**.

Het aangepaste Stappenplan (Focus op integratie in huidige Edit)

Fase 1: Architectuur van de Custom Editor

- Definieren welke AdminJS componenten we gaan gebruiken om de `contentRich` en `contentPlain` velden te "overriden".
- Bepalen hoe de logica werkt om het ene veld te verbergen als het andere wordt gekozen (Conditionele rendering).

Fase 2: Backend Mapping

- Zorgen dat wanneer je op "Save" klikt in deze nieuwe edit-view, AdminJS precies weet welke `id` hij moet updaten en de juiste kolommen (`contentRich` of `contentPlain`) met de nieuwe data moet vullen.

Fase 3: Implementatie (De Code)

1. **Custom Component maken:** Een React component dat de twee editors regelt.
2. **AdminJS Resource Configuratie:** De bestaande `EmailTemplates` resource aanpassen om te zeggen: "Gebruik voor het veld '`contentRich`' mijn nieuwe *HTML-editor component*."

Fase 4: Testen van de Workflow

1. Ga naar een template $\rightarrow$ Klik op **Edit**.
2. Kies via een switch: "Ik wil de tekst aanpassen". Controleer of de juiste editor verschijnt.
3. Wijzig content $\rightarrow$ Klik op **Save** $\rightarrow$ Controleer in de database (via Show mode) of de data correct is overgenomen.

Is dit de richting die je bedoelt? Dat we de huidige Edit-knop "slim" maken, zodat je niet meer door een lijst met velden hoeft te scrollen, maar direct naar de editor springt?

Q:

Ja. Het is dan nog te zien hoe wij de rich text in een htm layout op de monitor bekomen en via save op de pc zetten om dan het echte layout in de geedeteerde versie te kunnen archiveren. Maar het kan ook een pdf file zijn die van de html page wordt gemaakt. Dit laat toe de teksten de archiveeren of door te geven voor manueel taal check aan andere medewerkers van dit project.

Maar dan kan misschien ook in een aparte applicatie. Taal en template kiezen van de richt text of alles kiezen van de rich text en een browser oproepen om de richt text in html versie te bekijken. De ingevoerde teksten als `{{registration.firstname}}` kan zo blijven.

A:

Dit is een zeer slimme toevoeging aan het concept. Je bent nu eigenlijk aan het evolueren van een simpele "editor" naar een volledig **Content Management & Review Systeem**.

Je wilt dat de editor niet alleen bedoeld is om te *schrijven*, maar ook om te *controleren* (Preview) en te *archiveren/delen* (PDF/HTML export). Het behouden van de placeholders zoals `{{registration.firstname}}` is hierbij cruciaal: dit zijn je "variabelen".

Laten we het stappenplan uitbreiden met deze "**Review & Preview**" laag.

De Uitgebreide Workflow (Het nieuwe concept)

De interface moet drie functies krijgen:

1. **Edit:** De tekst aanpassen (HTML of Text).

2. **Preview (Browser/Live):** Een visuele weergave van de HTML-layout, waarbij de placeholders (`{{ . . }}`) zichtbaar blijven of vervangen worden door dummy-data (bijv: [Voornaam]) om te zien hoe het eruitziet.
 3. **Export (PDF/HTML):** De huidige versie "bevroren" in een bestand voor externe controle (taalcheckers) of archivering.
-

Het Verfijnde Stappenplan

Fase 1: De "Smart Editor" & Preview Logica

- **Placeholder Awareness:** We moeten ervoor zorgen dat de editor begrijpt dat `{{variable}}` geen fout is, maar een placeholder.
- **Live Preview Engine:** In plaats van een PDF te maken (wat vaak lastig is met complexe HTML/CSS), bouwen we een **Preview Window**. Dit is een kleine browser-component binnen je AdminJS pagina die de code rendert zonder de website zelf te veranderen.

Fase 2: De "Export & Review" Module

Dit lost jouw behoefte aan taalcheckers en archivering op:

- **HTML Export:** Genereert een los `.html` bestand van de huidige `contentRich`. Dit kan de medewerker naar een externe vertaler sturen.
- **PDF Generatie (Optioneel):** Gebruik een library zoals `html-pdf` of `puppeteer` op de backend om de HTML-layout direct in een PDF te zetten voor officieel archief.

Fase 3: Implementatie van de "Review Flow"

We maken de interface als volgt opgebouwd:

1. **Top Bar:** Selectie van Template + Taal (zoals besproken).
 2. **Middle Section (De Editor):** De twee tabs: `[Edit Code]` en `[Visual Preview]`.
 - In `Edit Code` zie je de HTML/Text.
 - In `Visual Preview` zie je direct het resultaat van de code.
 3. **Bottom Bar (Acties):**
 - `[Opslaan]` $\rightarrow$ Update database.
 - `[Download HTML]` $\rightarrow$ Download bestand voor vertaler.
 - `[Download PDF]` $\rightarrow$ Voor het archief.
-

Technische Uitdaging: De "Placeholder" Problematiek

Als je een preview maakt, wil je niet dat de tekst `{{registration.firstname}}` ziet als rauwe code in de preview. Je wilt dat de medewerker ziet waar die naam komt te staan.

De oplossing: In de Preview-modus vervangen we in de browser tijdelijk alle teksten tussen `{{` en `}}` door een visueel herkenbare placeholder, bijvoorbeeld: `[FIRSTNAME_HIER]` of `<span style="background: yellow;">[NAAM]</span>`.

Dit maakt het voor je taalcheckers (die misschien geen programmeur zijn) heel duidelijk wat de tekst is die zij moeten vertalen.

Hoe gaan we dit aanpakken in de code?

Als we straks naar de implementatie gaan, zullen we deze stappen volgen:

1. **Backend:** Een `export` endpoint maken dat een string (de HTML) accepteert en er een `.html` of `.pdf` file van maakt.
2. **Frontend:** De AdminJS component uitbreiden met een `Preview Tab`.

Vind je het goed als we dit stappenplan vastleggen? Zullen we daarna beginnen met de eerste technische stap: Het definiëren van de nieuwe API-structuur die nodig is om zowel te *lezen*, te *previewen* als te *exporteren*?